Software Enginering Project Report 2

02161 Software Engineering I
DTU Compute, Spring 2026

Group 16

Authors

Nicolai Søgaard -s234146
Nat Skovlund - s245203
Mathias Kjølbye - s240803
Vedanta Sinha - s245010
Viktor Munk - 240335

Git Repository:
`https://gitlab.gbar.dtu.dk/XXXX/software-project`
16. april 2026

Indhold

1 Introduction **1**
1.1 Purpose and Scope . . . 1
1.2 Relation to Report 1 . . . 1

2 Implemented System Overview **2**
2.1 Implemented Functionality . . . 2
2.2 Architecture Overview . . . 2
2.3 Non-Implemented Features . . . 2
2.4 Traceability to Report 1 . . . 2

3 Systematic Tests (White-Box Tests) **3**
3.1 Choice of Methods . . . 3
3.2 `ProjectService.createProject(...)` . . . 3
3.2.1 Method . . . 3
3.2.2 Input Properties . . . 3
3.2.3 Concrete Test Cases . . . 3
3.2.4 Implementation of Tests . . . 4
3.3 `ActivityService.createActivity(...)` . . . 4
3.3.1 Method . . . 4
3.3.2 Input Properties . . . 4
3.3.3 Concrete Test Cases . . . 4
3.4 `ActivityService.addDeveloperToActivity(...)` . . . 4
3.4.1 Method . . . 4
3.4.2 Input Properties . . . 5
3.4.3 Concrete Test Cases . . . 5
3.5 `TimeRegistrationService.registerTime(...)` . . . 5
3.5.1 Method . . . 5
3.5.2 Input Properties . . . 5
3.5.3 Concrete Test Cases . . . 6
3.6 `AbsenceService.registerAbsence(...)` . . . 6
3.6.1 Method . . . 6
3.6.2 Input Properties . . . 6
3.6.3 Concrete Test Cases . . . 6
3.7 Summary of White-Box Testing . . . 6

4 Design by Contract **8**
4.1 `ProjectService.createProject(...)` . . . 8
4.1.1 Method with Assertions . . . 8
4.1.2 Contract . . . 8
4.1.3 Discussion . . . 8
4.2 `ActivityService.createActivity(...)` . . . 8
4.2.1 Method with Assertions . . . 8

4.2.2 Contract . . . 8
4.3 `ActivityService.addDeveloperToActivity(...)` . . . 8
4.3.1 Method with Assertions . . . 8
4.3.2 Contract . . . 9
4.4 `TimeRegistrationService.registerTime(...)` . . . 9
4.4.1 Method with Assertions . . . 9
4.4.2 Contract . . . 9
4.5 `AbsenceService.registerAbsence(...)` . . . 9
4.5.1 Method with Assertions . . . 9
4.5.2 Contract . . . 9
4.6 Discussion of Contracts . . . 9

5 Code Coverage **10**
5.1 Purpose of Coverage Analysis . . . 10
5.2 Test Basis for the Coverage Measurement . . . 10
5.3 Coverage Diagram . . . 10
5.4 Coverage Results . . . 11
5.5 Interpretation of the Results . . . 12
5.6 Effect of the Additional Tests . . . 12
5.7 Limitations of Coverage . . . 12
5.8 Remaining Gaps . . . 12
5.9 Summary of Coverage Analysis . . . 13
5.10 Coverage Observations . . . 13

6 Design Patterns and SOLID Principles **14**
6.1 Layered Structure . . . 14
6.2 Single Responsibility Principle . . . 14
6.3 Dependency Inversion Principle . . . 14
6.4 Encapsulation . . . 14
6.5 Design Patterns . . . 14
6.5.1 Single Responsibility Principle . . . 15
6.5.2 Open/Closed Principle . . . 15
6.5.3 Dependency Inversion Principle . . . 15
6.6 Discussion . . . 15

7 Conclusion **16**
7.1 Reflection on the Development Process . . . 16
7.2 Collaboration and Contribution Weights . . . 16

8 Contribution Table **17**

9 Appendix **18**
9.1 Git Repository . . . 18
9.2 Run Instructions . . . 18

List of Figures **19**

List of Tables **20**

References **21**

1 Introduction

This report presents the final implementation and testing of our project management system. The system supports project creation, activity planning, assignment of developers, time registration, absence registration, project progress tracking, and reporting.

The purpose of this report is to document:

- systematic white-box testing of selected methods,
- design by contract for the same methods,
- code coverage after executing all tests,
- the use of design patterns and SOLID principles,
- and a final evaluation of the implemented system compared to the original design from Report 1.

We aim to maintain traceability between requirements, design, implementation, and tests.

1.1 Purpose and Scope

The purpose of this report is to document the final implementation of the system and to evaluate its quality. The report focuses on systematic testing, design by contract, code coverage, and the relationship between the implemented solution and the design presented in Report 1.

The main emphasis is placed on the core service-layer functionality, since this part contains the central application logic and is therefore most relevant for detailed analysis.

1.2 Relation to Report 1

This report builds on the analysis and design work presented in Report 1. Where Report 1 focused on requirements, use cases, and system design, the present report focuses on the implemented solution and its evaluation.

Some implementation details were adjusted during development to keep the scope manageable, but the final system still reflects the central ideas and requirements identified in the earlier report.

2 Implemented System Overview

This section briefly introduces the implemented system and its overall structure.

2.1 Implemented Functionality

The final system supports the following main features:

- creation of projects,
- creation and planning of activities,
- assignment of developers to activities,
- time registration on activities,
- absence registration,
- assignment of project leaders,
- project reporting,
- and progress tracking.

2.2 Architecture Overview

The implementation is structured into the following layers:

- **Domain layer**: core entities such as `Project`, `Activity`, `Developer`, `Absence`, and `TimeRegistration`,
- **Repository layer**: in-memory storage and lookup of system data,
- **Service layer**: application logic and coordination of operations,
- **User interface layer**: console and JavaFX-based interaction.

This layered design supports separation of concerns and improves testability.

2.3 Non-Implemented Features

2.4 Traceability to Report 1

3 Systematic Tests (White-Box Tests)

In this section, we present systematic white-box tests for selected methods. As required, we selected one method per group member. The chosen methods are action-oriented methods and not simple getters or setters.

3.1 Choice of Methods

The following methods were selected:

- `ProjectService.createProject(...)`
- `ActivityService.createActivity(...)`
- `ActivityService.addDeveloperToActivity(...)`
- `TimeRegistrationService.registerTime(...)`
- `AbsenceService.registerAbsence(...)`

These methods were chosen because they represent key system behaviour and contain validation logic, state changes, or coordination across multiple classes.

3.2 `ProjectService.createProject(...)`

3.2.1 Method

`[Insert method code here]`

3.2.2 Input Properties

Property	Description
Project name valid	Non-empty project name
Project name invalid	Empty or blank project name

Tabel 1: Input properties for `createProject(...)`

3.2.3 Concrete Test Cases

Test case	Input	Expected result
TC1	"WebShop"	Project is created successfully
TC2	""	Exception: project name cannot be empty

Tabel 2: Concrete test cases for `createProject(...)`

3.2.4 Implementation of Tests

The planned test cases were implemented as executable tests in the project. These tests verify both valid and invalid input behaviour.

3.3 `ActivityService.createActivity(...)`

3.3.1 Method

`[Insert method code here]`

3.3.2 Input Properties

Property	Description
Project exists	The given project ID exists in the system
Project does not exist	The project ID is unknown
Activity name valid	Non-empty activity name
Activity name invalid	Empty activity name

Tabel 3: Input properties for `createActivity(...)`

3.3.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Existing project, "Requirements"	Activity created
TC2	Unknown project, "Design"	Exception: project not found
TC3	Existing project, empty name	Exception: activity name cannot be empty

Tabel 4: Concrete test cases for `createActivity(...)`

3.4 `ActivityService.addDeveloperToActivity(...)`

3.4.1 Method

`[Insert method code here]`

3.4.2 Input Properties

Property	Description
Developer exists	The given developer initials exist
Developer does not exist	Unknown developer initials
Activity exists	The target activity exists in the chosen project

Tabel 5: Input properties for `addDeveloperToActivity(...)`

3.4.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Existing developer assigned to existing activity	Developer added successfully
TC2	Unknown developer assigned to existing activity	Exception: developer not found

Tabel 6: Concrete test cases for `addDeveloperToActivity(...)`

3.5 `TimeRegistrationService.registerTime(...)`

3.5.1 Method

`[Insert method code here]`

3.5.2 Input Properties

Property	Description
Hours valid	Positive number and multiple of 0.5
Hours zero or negative	Invalid number of hours
Hours not multiple of 0.5	Invalid granularity
Activity exists	The activity exists in the specified project

Tabel 7: Input properties for `registerTime(...)`

3.5.3 Concrete Test Cases

Test case	Input	Expected result
TC1	2.5 hours	Time registration stored successfully
TC2	0.0 hours	Exception: invalid number of hours
TC3	1.3 hours	Exception: must be multiple of 0.5

Tabel 8: Concrete test cases for `registerTime(...)`

3.6 `AbsenceService.registerAbsence(...)`

3.6.1 Method

`[Insert method code here]`

3.6.2 Input Properties

Property	Description
Developer is free in period	Absence can be registered
Developer is busy in period	Absence must be rejected
Valid week interval	Start and end period form a meaningful interval

Tabel 9: Input properties for `registerAbsence(...)`

3.6.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Vacation in free period	Absence registered successfully
TC2	Vacation in already booked period	Exception: vacation denied

Tabel 10: Concrete test cases for `registerAbsence(...)`

3.7 Summary of White-Box Testing

The systematic tests were derived from input partitions and boundary-relevant cases. The goal was to ensure that each selected method was tested for both normal behaviour and

relevant error conditions. These white-box tests complement the existing BDD tests and provide more explicit validation of implementation logic.

4 Design by Contract

This section presents contracts for the same methods used in the systematic testing section. For each method, we describe preconditions and postconditions and show how these are enforced using `assert` statements.

4.1 `ProjectService.createProject(...)`

4.1.1 Method with Assertions

`[Insert method with assert statements here]`

4.1.2 Contract

Precondition:

$$name \neq null \wedge name.trim().length() > 0$$

Postcondition:

$$result \neq null \wedge result.getName() = name$$

4.1.3 Discussion

The precondition ensures that only valid project names are accepted. The postcondition guarantees that a project object is created and stored consistently.

4.2 `ActivityService.createActivity(...)`

4.2.1 Method with Assertions

`[Insert method with assert statements here]`

4.2.2 Contract

Precondition:

$$projectExists(projectId) \wedge activityName \neq null \wedge activityName.trim().length() > 0$$

Postcondition:

$$activity \in project.activities \wedge activity.getName() = activityName$$

4.3 `ActivityService.addDeveloperToActivity(...)`

4.3.1 Method with Assertions

`[Insert method with assert statements here]`

4.3.2 Contract

Precondition:

$$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(initials)$$

Postcondition:

$$developer \in activity.assignedDevelopers$$

4.4 TimeRegistrationService.registerTime(...)

4.4.1 Method with Assertions

```
[Insert method with assert statements here]
```

4.4.2 Contract

Precondition:

$$hours > 0 \wedge hours \bmod 0.5 = 0$$

Postcondition:

$$\exists r \in activity.timeRegistrations : r.hours = hours$$

4.5 AbsenceService.registerAbsence(...)

4.5.1 Method with Assertions

```
[Insert method with assert statements here]
```

4.5.2 Contract

Precondition:

$$developerExists(initials) \wedge periodIsValid(startWeek, startYear, endWeek, endYear)$$

Postcondition:

$$absenceRegistered \vee exceptionThrown$$

4.6 Discussion of Contracts

The contracts make assumptions and guarantees explicit. They improve readability, support debugging, and document the intended behaviour of the methods. In our implementation, assertions were added to enforce preconditions and postconditions directly in code.

5 Code Coverage

This section presents the code coverage results obtained after executing the full automated test suite. The purpose of the coverage analysis is to evaluate how much of the implementation is exercised by the tests and to identify parts of the system that are insufficiently tested.

Code coverage does not by itself prove correctness, but it is a useful indicator of how thoroughly the test suite explores the implementation. In this project, coverage was used as a supplement to the systematic white-box tests and the existing BDD-based tests.

5.1 Purpose of Coverage Analysis

The purpose of the coverage analysis was to assess which parts of the implementation were exercised during testing and to identify areas where additional tests were needed.

A high coverage percentage does not guarantee that the system is free of defects, but low coverage often indicates that important logic or exceptional behaviour has not been tested. For this reason, the coverage results were used as a diagnostic tool to improve the quality of the test suite.

5.2 Test Basis for the Coverage Measurement

The coverage results were obtained after running all automated tests in the project. These tests include both the previously implemented BDD scenarios and the additional systematic unit tests introduced for this report.

The BDD tests mainly cover end-to-end usage scenarios and validate the overall behaviour of the system from a user-oriented perspective. The additional white-box tests focus more directly on method-level logic, including validation, exceptional cases, and internal state changes in the service layer.

Together, these two testing approaches provide broader and more meaningful coverage than either approach alone.

5.3 Coverage Diagram

Figure 1 shows the code coverage for the individual Java files after executing the full test suite.

figures/code_coverage.png

Figur 1: Code coverage after executing all automated tests

5.4 Coverage Results

The coverage results show that the service layer achieves the strongest coverage. This is expected, since the most important application logic is implemented in service classes, and these classes are exercised by both the BDD tests and the additional white-box tests.

The domain layer has more moderate coverage. Many domain classes mainly represent data and simple state transitions, and therefore contain less complex behaviour to test. As a result, lower coverage in these classes is not necessarily problematic.

The user interface layer has the lowest coverage. This is also expected, since automated tests in this project were primarily focused on domain and service logic, rather than JavaFX-related interaction code. UI code is generally harder to test automatically and was therefore not the main priority in the test strategy.

5.5 Interpretation of the Results

The coverage analysis indicates that the most important business logic is tested to a reasonable extent. In particular, methods related to project creation, activity management, developer assignment, time registration, and absence handling are covered by automated tests.

The analysis also shows that coverage is not evenly distributed across the system. This is acceptable, since not all parts of the implementation have the same importance or complexity. In this project, priority was given to testing state-changing methods and methods containing validation logic, because faults in these areas would have the greatest impact on system correctness.

5.6 Effect of the Additional Tests

The additional systematic tests introduced for this report improved coverage especially in the service layer. These tests were designed to exercise both normal execution paths and important error conditions, such as invalid input, missing entities, and conflicting registrations.

Compared to relying only on BDD scenarios, the added unit tests make it possible to test individual methods more precisely. This increases coverage of branches that would otherwise be difficult to reach through end-to-end scenarios alone.

In particular, the added tests strengthened coverage of:

- input validation,
- exception handling,
- boundary cases,
- and methods that modify the internal state of the system.

5.7 Limitations of Coverage

Although coverage is a useful metric, it has clear limitations. A line can be executed during a test without the test actually verifying the correctness of the result. Therefore, coverage must not be interpreted as proof that the implementation is correct.

For this reason, coverage was not used as the sole quality criterion. Instead, it was combined with systematic test design, assertions about expected behaviour, and discussion of preconditions and postconditions in the design by contract section.

5.8 Remaining Gaps

The coverage analysis revealed several areas where testing could still be improved. These include:

- UI-specific behaviour,
- repository and persistence-related behaviour,
- rare exceptional cases,
- and interactions that depend on longer execution flows.

These parts were not prioritised in the current project, because the main focus was on validating the core service logic. However, they would be relevant targets for future test expansion.

5.9 Summary of Coverage Analysis

Overall, the coverage results support the conclusion that the most critical parts of the system are exercised by automated tests. The strongest coverage is found in the service layer, where most of the application logic resides, while lower coverage in the UI layer is acceptable given the chosen scope and test strategy.

The coverage analysis also helped identify weaknesses in the test suite and motivated the addition of focused white-box tests. In this way, code coverage served both as an evaluation tool and as a guide for improving test quality.

5.10 Coverage Observations

Layer / Module	Coverage tendency	Interpretation
Service layer	High	Core business logic is tested directly through both BDD and unit tests
Domain layer	Medium	Contains simpler data-oriented classes with limited logic
UI layer	Low	UI behaviour was not the primary focus of automated testing
Exceptional branches	Improved	Additional white-box tests increased coverage of validation and error handling

Tabel 11: Main observations from the coverage analysis

6 Design Patterns and SOLID Principles

This section discusses the design principles and patterns used in the implementation.

6.1 Layered Structure

The system is divided into domain, repository, service, and UI layers. This supports the **Single Responsibility Principle**, since each layer has a clearly defined purpose.

6.2 Single Responsibility Principle

Several classes have one main responsibility:

- domain classes represent data and core concepts,
- repository classes manage storage and lookup,
- service classes implement application logic,
- UI classes handle interaction with the user.

6.3 Dependency Inversion Principle

The project description explicitly encouraged the use of dependency inversion for in-memory storage. Our design partially supports this idea by separating repositories from services. However, the implementation still depends on concrete repository classes rather than interfaces. This means the design moves in the direction of dependency inversion, but does not fully realise it.

6.4 Encapsulation

The domain classes encapsulate state and expose behaviour through methods. For example, project progress and total registered hours are calculated through dedicated methods instead of direct field access.

6.5 Design Patterns

No formal GoF design pattern was heavily required in this project. The most pattern-like structure in the implementation is the use of repositories and services, which resembles a simple service-repository architecture. We chose this structure because it keeps the code understandable and fits the scale of the system.

6.5.1 Single Responsibility Principle

6.5.2 Open/Closed Principle

6.5.3 Dependency Inversion Principle

6.6 Discussion

The design choices were guided mainly by simplicity, readability, and traceability to the requirements. Where more advanced patterns were not used, this was a deliberate choice to avoid unnecessary complexity.

7 Conclusion

The final system implements the core functionality required for project planning, activity management, developer assignment, time registration, absence handling, and project tracking.

The implemented product is consistent with the main requirements selected in Report 1, although some parts were simplified to keep the scope manageable. In particular, the final implementation focuses on a coherent in-memory solution with automated tests and a working user interface.

Compared to the design in Report 1, the final implementation follows the same main structure, but some adjustments were made during implementation. These changes improved feasibility and reduced complexity while preserving the key system behaviour.

The testing work consists of both BDD-style feature tests and additional systematic white-box tests. Together, these provide stronger confidence in the correctness of the core application logic.

7.1 Reflection on the Development Process

The project was developed collaboratively across the group. Work was divided across analysis, implementation, testing, and documentation tasks. During the project, design decisions were refined as implementation details became clearer. This led to a final solution that is more concrete and implementation-oriented than the original design in Report 1.

7.2 Collaboration and Contribution Weights

Table 12 shows the contribution weights for each group member.

s240803	s234146	s245203	s245010	s240335
1.0	1.0	1.0	1.0	1.0

Tabel 12: Contribution weights for the group members

8 Contribution Table

Student Name	Student ID	Contribution Weight
[INSERT NAME]	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE]

Tabel 13: Contribution weights for the group members.

9 Appendix

9.1 Git Repository

The Git repository for the project is available at:

`[Insert repository URL here]`

9.2 Run Instructions

After unpacking the ZIP archive, the project can be imported as a Maven project in Eclipse, IntelliJ IDEA, or Visual Studio Code.

Run application

```
mvn javafx:run
```

Run tests

```
mvn test
```

The system includes at least one developer with the initials `huba`, as required in the project description.

Figurer

1 Code coverage after executing all automated tests 11

Tabeller

1 Input properties for `createProject(...)` 3
2 Concrete test cases for `createProject(...)` 3
3 Input properties for `createActivity(...)` 4
4 Concrete test cases for `createActivity(...)` 4
5 Input properties for `addDeveloperToActivity(...)` 5
6 Concrete test cases for `addDeveloperToActivity(...)` 5
7 Input properties for `registerTime(...)` 5
8 Concrete test cases for `registerTime(...)` 6
9 Input properties for `registerAbsence(...)` 6
10 Concrete test cases for `registerAbsence(...)` 6
11 Main observations from the coverage analysis 13
12 Contribution weights for the group members 16
13 Contribution weights for the group members. 17

Litteratur